

C++ & QT Day3-hw2 과제 보고서

로봇 21기 예비단원 최윤서

1. 과제 조건 및 이해

- 1) 맵 사이즈 구현
- 2) 수동 장애물, 시작점, 도착점 설정
- 3) 탐색 과정 시각화

2. 코드 구현

2-1. 맵 사이즈 구현

- 라디오버튼으로 맵 사이즈 선택 -> createGrid 함수 사용 -> 가로 세로 픽셀 수 만큼 격자점 생성

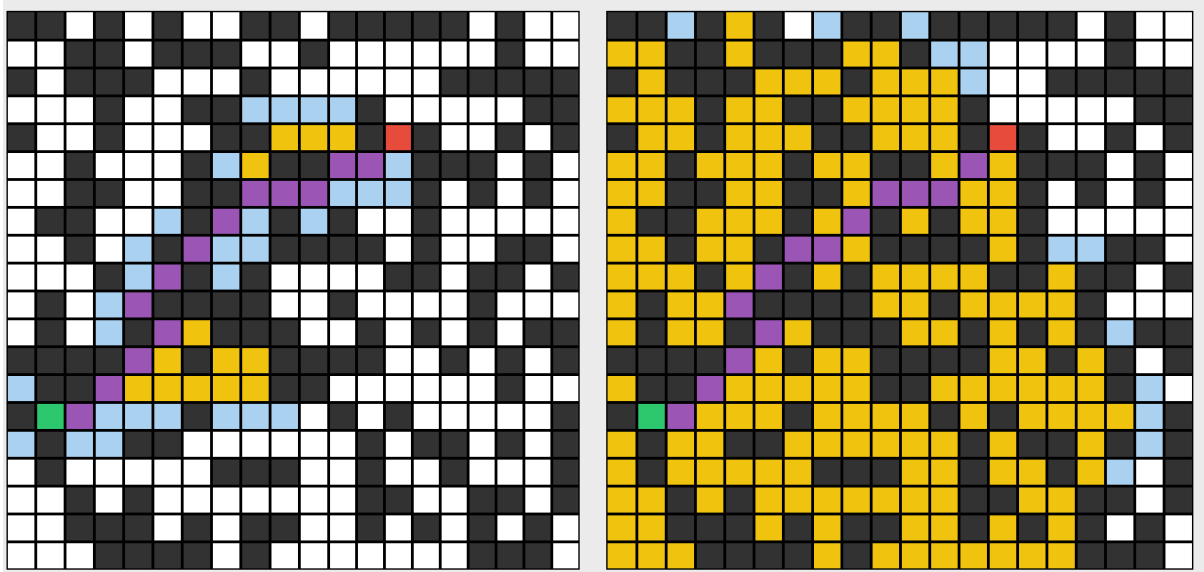
```
if (ui->radioButton_5->isChecked()) {  
    rows = 10; cols = 10;  
    ...  
    createGrid(ui->gridLayout_2, cellsAStar, rows, cols); // A* 쪽  
    createGrid(ui->gridLayout_3, cellsDijkstra, rows, cols); // Dijkstra 쪽
```

2-2. 수동 장애물 생성, 시작점, 도착점 설정

- 정해진 픽셀 가로 세로 사이즈에 따라 셀 타입 반영 -> 마우스가 셀을 클릭하는 것을 인식하기 위함
- 각 단계마다 Apply 버튼을 따로 만들어서 단계별로 실행할 수 있도록함
-> 도착지 설정인지 시작점 설정인지 장애물 설정인지 클릭만으로 알 수 없기 때문에

2-3. 탐색 과정 시각화

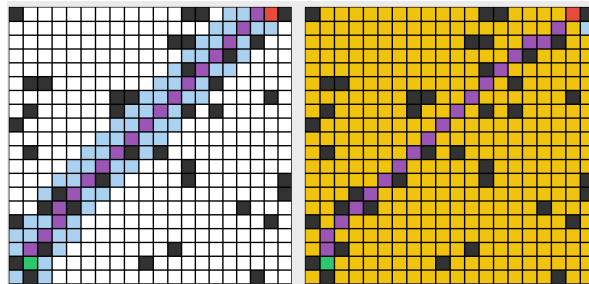
- 하늘색: Open List, 아직 최종 확정은 안됐지만 다음에 방문할 후보 중 하나
- 노란색: Close List, 이미 방문 완료한 칸들
- 보라색: 최종 경로



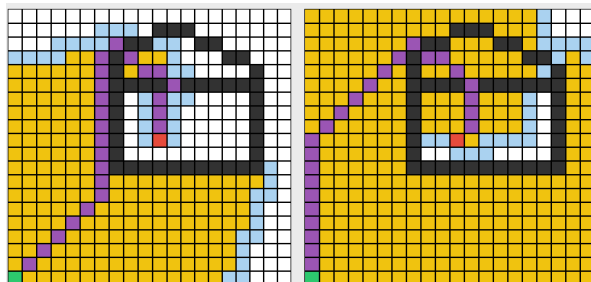
3. 분석 및 비교

3-1. 최종 경로 비용

- 장애물 10% 일 때: A* 알고리즘이 압도적으로 빠름



- 장애물이 대체적으로 막혀있을 때: 위의 경우보단 아니지만 A*가 더 빨랐음



3-2. 탐색한 노드 수

- 일반적인 경우: A* 알고리즘이 탐색한 노드 수가 훨씬 적음
- 3-1의 경우처럼 장애물이 대체적으로 막혀있다면 A*알고리즘도 탐색하는 노드 수가 급격하게 많아짐

3-3. 실행 시간

- 장애물이 사방에 막혀있던, 일반적인 경우이건 거의 모든 경우 A*가 더 빨랐음
(약 90% 이상 빠르고 10%는 비슷비슷하지만 A*가 미세하게 더 빠름)

4. A*와 Dijkstra 코드 구현

4-1. 공통 구조

- 우선순위 큐에 {우선순위값, row, col}을 넣고 매번 가장 작은 우선순위 값을 가진 셀을 꺼내서 처리
- 한번 꺼내진 셀은 close list로 표시
- QTimer -> 한번에 노드 1개씩만 처리함 -> 파란색이 퍼져나가는 애니메이션

4-2. 다익스트라

- 우선순위 값 g값만 계산
- 목적지 방향을 전혀 모르고 무조건 지금까지 가장 가까운곳부터 확장
-> 따라서 사방으로 고르게 퍼져나감

4-3. A*

- 우선순위값 = $g + h$ 값
- 지금까지 온 거리 + 앞으로 가야할 거리 -> 우선순위 매김